

DÉVELOPPEMENT AUGMENTÉ · INITIATION

Premiers pas vers le développement augmenté par l'IA

Une méthode concrète pour travailler avec des agents de code

demande



changement



vérification

PROMESSE

Une heure pour repartir avec une méthode

En une heure, vous ne deviendrez pas expert des agents de code.

Vous repartirez avec une méthode que vous pourrez essayer dès demain.

Modèle, assistant, agent

MODÈLE

Répond

Produit une réponse à partir des informations reçues.

ASSISTANT

Suggère

Aide dans l'éditeur, le chat ou le terminal.

AGENT

Agit

Explore, utilise des outils, modifie et vérifie.

WORKFLOW

Les étapes importantes restent visibles



Le plan est un point de contrôle humain, pas une formalité.

Comprendre → planifier → approuver → agir → vérifier → revoir

04 · 2 min

CONTEXTE

La qualité de la réponse dépend du contexte disponible





LIVE DEMO · 4 min max

Démonstration 1 — comprendre avant de modifier

Résultat attendu : architecture, point d'entrée, tests, commandes, rules et skills identifiés

RULES

Les rules rendent les attentes persistantes

SANS RULES

réponse générique

conventions oubliées

contrôles non précisés

Ce que le projet exige



AVEC RULES

architecture respectée

tests explicitement
attendus

compte-rendu
standardisé

07 · 2 min

SKILLS

Une rule dit quoi respecter. Un skill décrit comment agir.

RULE

Ce que le projet exige

Architecture · tests · sécurité ·
restitution

Les skills encapsulent une méthode réutilisable

SKILL

Comment réaliser une tâche

Lire · planifier · implémenter ·
contrôler · rendre compte

08 · 2 min



LIVE DEMO · 4 min max

Démonstration 2 — obtenir un plan avant d'obtenir du code

Résultat attendu : critères, fichiers, tests, risques et point d'approbation humaine

Un petit changement, visible avant et après

AVANT

```
@app.get("/tasks")
def list_tasks():
    return tasks
```



APRÈS

```
@app.get("/tasks")
def list_tasks(status: TaskStatus |
None = None):
    if status is None:
        return tasks
    return [task for task in tasks
            if task.status ==
status]
```

CONTRÔLES DÉTERMINISTES

Le résumé de l'agent ne remplace pas les contrôles

- ✓ `pytest`
- ✓ `ruff check .`
- ✓ `ruff format --check .`
- ✓ `mypy app`

Contrôle réussi

Une preuve valide des commandes et du diff.
Une sortie réellement exécutée.

Non exécuté $\neq$ réussi

Un manque de preuve reste une limite.

11 · 4 min

HARNAIS

Plus l'agent peut agir, plus le harnais doit être explicite

05	Revue humaine	diff · critères · décision
04	Tests, linter, CI	résultats déterministes
03	Outils et permissions	agir dans un périmètre borné
02	Rules et skills	attentes + méthode
01	Contexte	code · demande · contraintes

12 · 3 min



LIVE DEMO · 3 min max

Démonstration 4 — comparer le résultat à la demande

Résultat attendu : demande, plan, rules, diff et résultats comparés sans modifier de fichier

FRAMEWORKS

Un framework formalise un workflow

SPEC KIT

spec → plan →
tasks →
implement /
converge

Pour structurer
l'intention.

OPENSPEC

explore →
propose →
apply → archive

Pour aligner
avant de
construire.

BMAD METHOD

Workflows
adaptatifs par
rôles

Pour
industrialiser
progressivement.

AIDD

Cycle logiciel
plus large

Skills, agents,
règles et revue.

SYNTHÈSE

Commencer petit. Renforcer progressivement le harnais.

01

Donner du
contexte

02

Écrire les
rules

03

Utiliser des
skills

04

Approuver
le plan

05

Vérifier le
résultat

L'agent propose et exécute ; le développeur reste responsable.

8 min formations existantes · 10 min questions · TDD : piste intéressante, pas obligation ^{15 · 4 min}